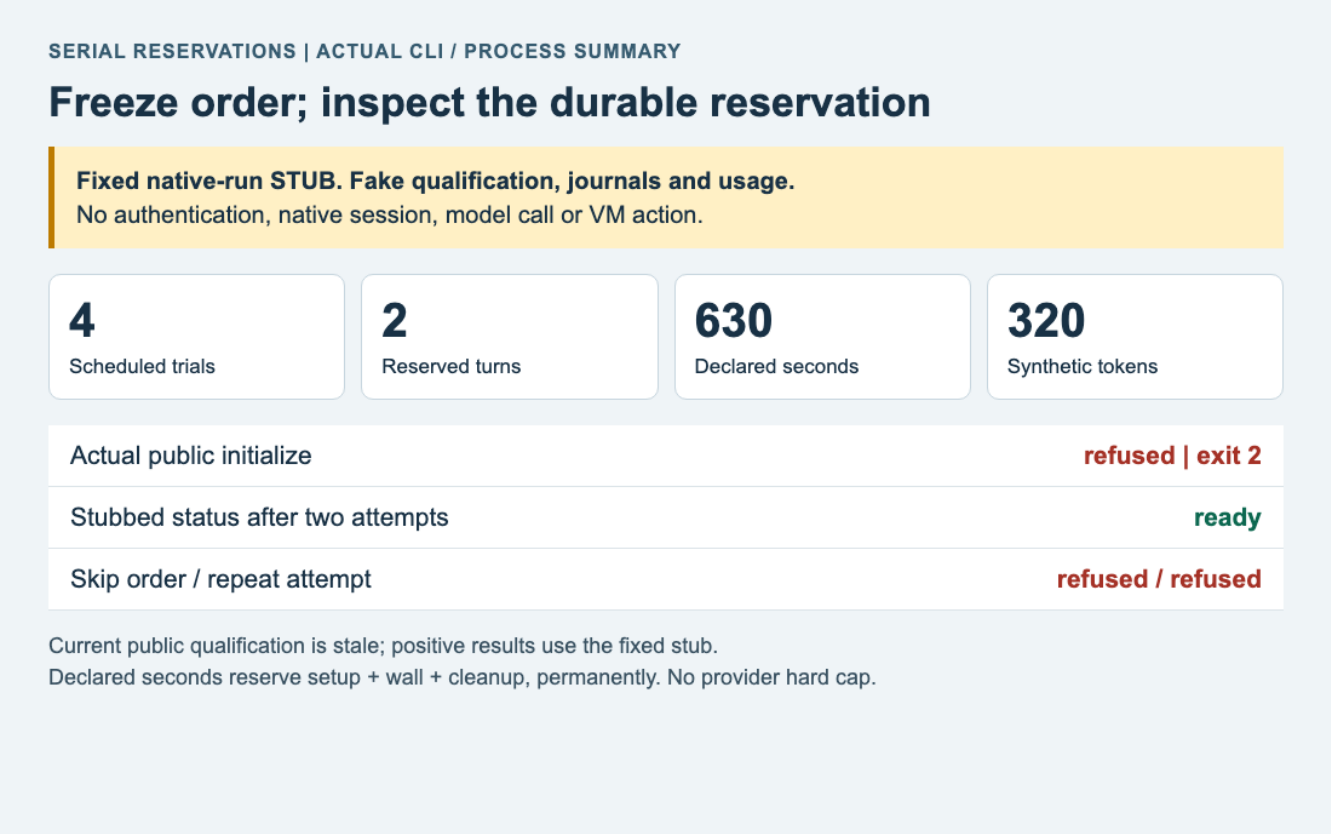
Serial native trial reservations

Evaluation-accounting workflow on Python 3.11+. Browser-rendered actual CLI/process summaries use a fixed native-run STUB and fake qualifications, journals and usage. Not terminal images or native/model evidence.

1. Initialize and follow the frozen order

Read the serial contract before using trial_reservation.py initialize, status or run-next. Supply the frozen stdin context and private execution root. Current public qualification refuses initialization; the positive example uses a fixed test-only native-run stub.



Renewed smoke at source checkpoint b57ea16629b1: actual public refusal and actual CLI main through a fixed stub. The checkpoint existed before this run; thirteen runner and three grader source hashes match.

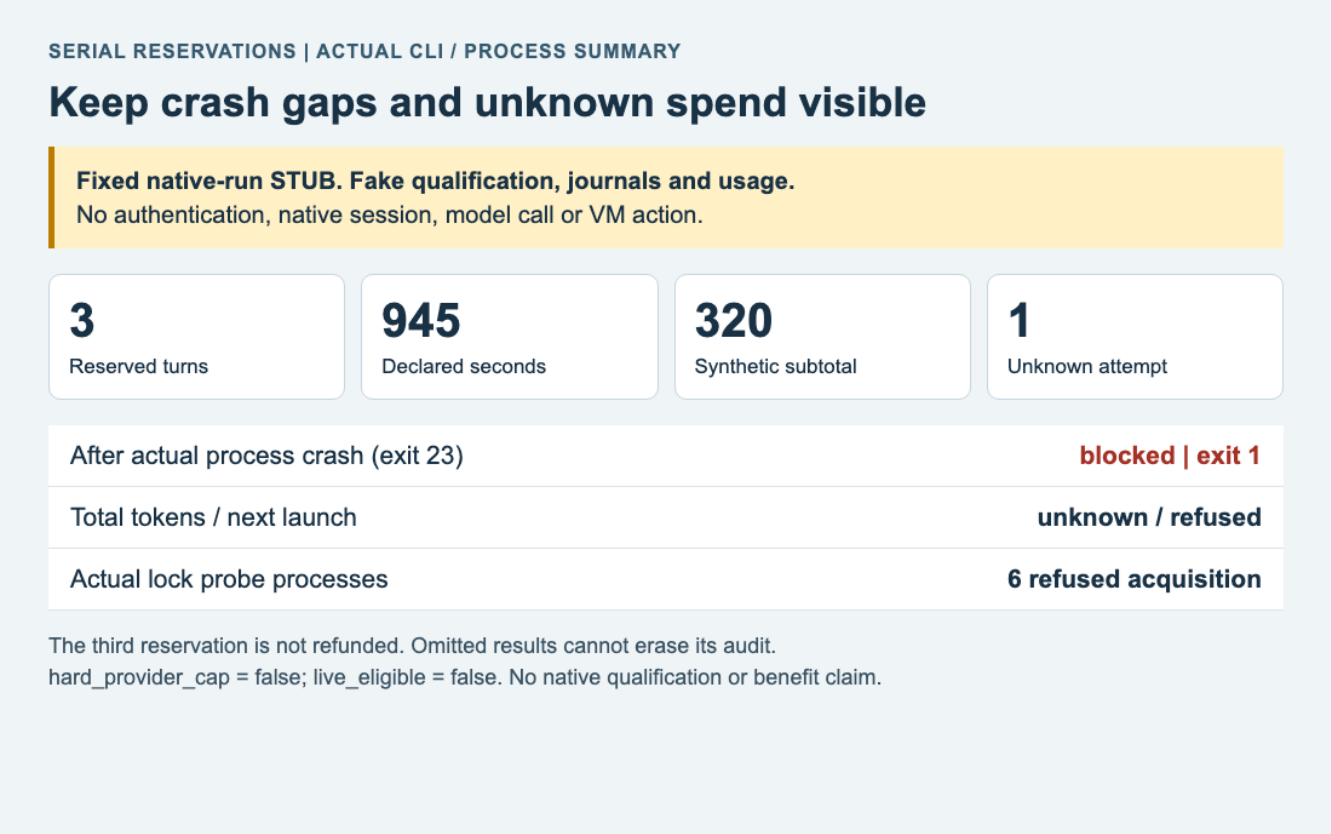
Expected result: After two stubbed attempts: two reserved turns, 630 declared seconds and 320 synthetic tokens. Wrong order and retry refuse. These are initial-turn and permanent phase-time reservations, not provider request counts.

Serial native trial reservations

Evaluation-accounting workflow on Python 3.11+. Browser-rendered actual CLI/process summaries use a fixed native-run STUB and fake qualifications, journals and usage. Not terminal images or native/model evidence.

2. Audit interruption without a refund

Inspect status after interruption and retain the original reservation. Check execution_accounting when completion references omit an attempt. Unknown or incomplete facts prevent another launch; reconcile exact owned state rather than restarting the experiment to erase it.



A real local process exited 23 after reserving. The native runner and lifecycle remain synthetic. Separate processes confirmed the experiment lock stayed busy before task and after stubbed cleanup.

Expected result: Three reservations and 945 declared seconds remain. Synthetic 320 tokens are known; total is unknown and the next launch refuses. Post-use soft token checks may overshoot; hard_provider_cap and live_eligible stay false.